Guion de exposición: Consumo REST desde C# Windows Forms

Sistemas Distribuidos (SIS-258) · Grupo 3 · Jhonn Llanos, Josías [apellido], Gabriel [apellido]

1. Reparto y tiempos

Quién	Diapositivas	Tema	Tiempo
Josías	1 a 4	Presentación, qué es REST, códigos de respuesta y arquitectura	5 min
Gabriel	5 a 8	Login con JWT, el JSON que viaja, endpoints y permisos	5 min
Jhonn	9 a 14	Código del cliente, demostración en vivo y cuestionario	5 a 6 min
Total			**≈ 15 min**

El texto está calculado para unos 5 minutos hablando a ritmo normal (unas 130 palabras por minuto). No hace falta leerlo palabra por palabra: es la idea de lo que hay que decir. El mismo texto está en las notas de cada diapositiva.

Antes de empezar (lo hace Jhonn, 5 minutos antes): la API corriendo con `php artisan serve`, el cliente abierto con `dotnet run` en la ventana de login y el código QR del cuestionario ya pegado en la diapositiva 14.

2. Guion de Josías (diapositivas 1 a 4)

Duración aproximada: 709 palabras, unos 5 minutos.

Diapositiva 1 · Portada

> Buenos días, ingeniero; buenos días, compañeros. Somos el grupo 3: Jhonn Llanos, Gabriel y yo, Josías. Hoy les vamos a mostrar cómo consumimos una API REST desde una aplicación de escritorio hecha en C# con Windows Forms.

> La API es la de elecciones que nos pasó el ingeniero, hecha en Laravel. Tiene dos partes: un login que nos da acceso y un CRUD de candidatos, es decir, listar, crear, editar y eliminar candidatos. Nuestro trabajo fue construir el cliente: una ventana donde el usuario inicia sesión y después administra los candidatos sin escribir ni una línea de SQL.

> La exposición está dividida en tres partes. Yo voy a explicar qué significa consumir una API REST y cómo está armado el sistema. Después Gabriel explica la seguridad: el login con token y los permisos. Al final Jhonn muestra el código y lo vemos funcionando en vivo.

Diapositiva 2 · Qué es consumir una API REST

> Empecemos por lo básico: ¿qué es consumir una API REST? Una API REST es un servidor que expone sus datos a través de direcciones web, igual que una página, pero en lugar de devolver una página con diseño devuelve datos puros en formato JSON. Consumirla significa que nuestro programa le hace pedidos por HTTP y usa lo que le responde.

> Cada pedido lleva un verbo que dice qué queremos hacer. GET es para leer, por ejemplo, listar todos los candidatos. POST es para crear uno nuevo. PUT es para modificar uno que ya existe. Y DELETE es para eliminarlo. Con esos cuatro verbos se arma cualquier CRUD.

> Lo más interesante es lo que dice abajo: a la API no le importa quién le pregunta. Un navegador con JavaScript, un programa en Java, uno en PHP, uno en Node o nuestra ventana en C# usan exactamente las mismas direcciones y reciben el mismo JSON. Por eso el ingeniero dividió la clase en grupos: la API es una sola y cada grupo hizo un cliente distinto. Eso es justamente lo que hace útil a REST: el servidor y el cliente están desacoplados, cada uno puede estar en un lenguaje diferente.

Diapositiva 3 · Códigos de respuesta

> Cuando hacemos un pedido, la API no solo devuelve datos: también devuelve un número, el código de estado, que dice cómo salió todo. Este número es muy importante para nosotros porque la ventana lo usa para decidir qué mensaje mostrar.

> Los que empiezan con 2 significan éxito. El 200 es el más común: todo bien, aquí están los datos; lo recibimos al listar o al editar. El 201 significa que se creó un registro nuevo, y es lo que devuelve la API cuando guardamos un candidato. El 204 significa que la operación salió bien pero no hay nada que devolver, y es lo que llega al eliminar.

> Los que empiezan con 4 son errores del lado del cliente. El 401 quiere decir no sé quién eres: pasa cuando pedimos algo sin haber iniciado sesión. El 403 es distinto: la API sí sabe quién eres, pero no tienes permiso para esa acción; Gabriel lo va a explicar con los roles. Y el 404 significa que pedimos algo que no existe, por ejemplo un candidato con un id que ya fue eliminado.

Diapositiva 4 · Arquitectura del sistema

> Ahora veamos cómo está armado nuestro sistema. Son tres piezas que se comunican por la red.

> La primera, a la izquierda, es nuestro cliente: la aplicación de Windows Forms escrita en C#. Para hablar con la API usa una clase de .NET llamada HttpClient, que sabe hacer pedidos GET, POST, PUT y DELETE.

> La segunda pieza, en el centro, es la API de Laravel. Corre en nuestra máquina en el puerto 8000. Ella recibe los pedidos, revisa el token, aplica las reglas de permisos y responde en JSON.

> La tercera pieza es la base de datos MySQL, llamada eleccion_api, donde está la tabla de candidatos.

> Lo importante de este diagrama es que el cliente nunca toca la base de datos directamente. No tiene usuario de MySQL ni escribe consultas SQL. Todo pasa por la API. Eso es más seguro, porque las reglas están en un solo lugar, y además es más ordenado: si mañana cambian la base de datos, nuestro cliente sigue funcionando igual mientras la API responda lo mismo.

> Con eso termino mi parte. Ahora Gabriel les explica cómo funciona la seguridad con el login.

3. Guion de Gabriel (diapositivas 5 a 8)

Duración aproximada: 731 palabras, unos 5 minutos.

Diapositiva 5 · El login entrega un token JWT

> Gracias, Josías. Antes de poder usar el CRUD, el usuario tiene que iniciar sesión, y para eso la API usa JWT, que significa JSON Web Token. El proceso tiene tres pasos.

> Primero, el cliente le pide un token a la API. Hace un POST a la dirección /api/login y le manda el email y la contraseña.

> Segundo, la API busca al usuario y compara la contraseña. Si todo es correcto, genera un token: una cadena de texto larga, firmada con una clave secreta que solo conoce el servidor. Ese token es como un pase de entrada y, en esta API, dura veinte horas.

> Tercero, el cliente guarda ese token y lo manda en todos los pedidos siguientes, en un encabezado llamado Authorization, con la palabra Bearer adelante. Bearer significa portador: quien porta este pase tiene acceso.

> ¿Por qué se hace así y no mandando la contraseña cada vez? Porque la contraseña viaja una sola vez, y después la API solo verifica la firma del token, que es mucho más rápido y seguro.

> Si un pedido llega sin token, o con un token falso o vencido, la API responde 401 y no deja pasar.

Diapositiva 6 · Una petición y su respuesta

> Para que se vea concreto, esta es exactamente la información que viaja por la red durante el login.

> A la izquierda está lo que manda nuestro cliente: un POST a /api/login, con un encabezado que dice que el contenido es JSON, y en el cuerpo un objeto con dos campos, email y password.

> A la derecha está lo que responde la API cuando los datos son correctos, con el código 200. Viene un mensaje que dice que se logró autenticar, el token, que es esa cadena larga que empieza con eyJ, el tipo, que es bearer, y los datos del usuario, donde se ve que su rol es admin.

> Nuestro cliente solo necesita un dato de toda esta respuesta: el token. Lo saca del JSON y lo guarda. Jhonn les va a mostrar la línea exacta del código donde hacemos eso.

> Si la contraseña fuera incorrecta, la API respondería con el código 400 y un mensaje que dice contraseña inválida, y la ventana le avisaría al usuario.

Diapositiva 7 · Las cinco llamadas del cliente

> Estas son todas las llamadas que hace nuestro cliente. Son cinco.

> La primera es el login que acabamos de ver: POST a /api/login, y si sale bien recibimos 200 y el token.

> Las otras cuatro son el CRUD de candidatos, y todas llevan el token. Para listar hacemos GET a /api/candidatos y recibimos 200 con la lista completa en JSON. Para crear hacemos POST a la misma dirección, mandando los datos del candidato, y recibimos 201. Para modificar hacemos PUT a /api/candidatos con el id del candidato al final de la dirección, y recibimos 200. Y para eliminar hacemos DELETE a la misma dirección con el id, y recibimos 204.

> Fíjense en que la dirección de listar y la de crear es la misma; lo que cambia es el verbo. Eso es REST: la dirección dice sobre qué recurso trabajamos y el verbo dice qué hacemos con él.

> Cada candidato tiene cuatro datos: nombres, apellidos, cargo, por ejemplo Rector o Decano, y ámbito, por ejemplo Universidad o Facultad. Y como dice la tabla, crear, modificar y eliminar solo los puede hacer un administrador.

Diapositiva 8 · Dos usuarios, dos niveles de acceso

> Eso nos lleva a los permisos. La API tiene dos tipos de usuario.

> El administrador, que en nuestras pruebas es test@example.com, puede hacer todo: listar, crear, editar y eliminar candidatos.

> El usuario normal, que es test2@example.com, solo puede ver la lista. Si intenta crear, editar o eliminar, la API le responde 403.

> ¿Cómo sabe la API qué rol tiene cada uno? El rol viaja dentro del token. Cuando la API generó el token en el login, metió adentro el id, el email y el rol del usuario. En cada pedido, el middleware de la API abre el token, verifica la firma y lee el rol. Si la acción es crear, editar o eliminar y el rol no es admin, corta ahí y responde 403.

> Aquí se ve clara la diferencia entre los dos errores que explicó Josías: 401 es cuando no hay token o es inválido, o sea, la API no sabe quién eres; 403 es cuando el token es válido, la API sí sabe quién eres, pero no te deja hacer eso.

> En la demostración vamos a probar los dos usuarios. Ahora Jhonn les muestra el código.

4. Guion de Jhonn (diapositivas 9 a 14)

Duración aproximada: 688 palabras, unos 5 minutos.

Diapositiva 9 · Los cinco archivos del proyecto

> Gracias, Gabriel. Ahora vamos al código. Todo el cliente está en cinco archivos y cada uno hace una sola cosa.

> Api.cs es el más importante: ahí están todas las llamadas HTTP, y ningún otro archivo hace pedidos a la red. FormLogin.cs es la ventana del email y la contraseña, y FormCandidatos.cs es la del CRUD, con la tabla y los botones. Candidato.cs es el molde del JSON: cuando la API manda un candidato, .NET llena solo un objeto de esta clase. Y Program.cs arranca la aplicación con el login.

> Las ventanas no saben nada de HTTP: solo llaman a Api. Así, si la API cambia de dirección, se toca una sola línea. En total son unas doscientas líneas, sin librerías extra.

Diapositiva 10 · Api.cs: el login

> Este es el método Login de Api.cs. Arriba creamos un solo HttpClient para toda la aplicación, con la dirección base de la API.

> La primera línea hace el POST: PostAsJsonAsync convierte el email y la contraseña a JSON y los manda; await significa que esperamos la respuesta sin congelar la ventana. Si el código no es de éxito, por ejemplo un 400, devolvemos false.

> Si salió bien, ReadFromJsonAsync saca el token del JSON, y la última línea es la clave: guardamos el token en el encabezado Authorization, con la palabra Bearer. Como el HttpClient es uno solo, desde aquí todos los pedidos del CRUD llevan el token automáticamente.

> Listar es una sola línea: GetFromJsonAsync hace el GET y convierte el JSON directo en una lista de candidatos.

Diapositiva 11 · FormLogin.cs

> Esta es la ventana de login. Fíjense en que no tiene nada de HTTP: solo llama a Api.Login con lo que el usuario escribió en las dos cajas de texto.

> Si el login devuelve verdadero, escondemos la ventana de login, abrimos la ventana de candidatos con ShowDialog y, cuando el usuario la cierra, cerramos también el login, y con eso termina la aplicación.

> Si devuelve falso, mostramos el mensaje Email o contraseña incorrectos.

> Y todo está dentro de un try y un catch. Esto es importante en un sistema distribuido: el servidor está en otro proceso y puede estar apagado. Si la API no responde, en lugar de que el programa se cierre con un error, el catch muestra un mensaje claro: no se pudo conectar con la API, revise que esté corriendo.

Diapositiva 12 · FormCandidatos.cs: el botón Guardar

> Esta es la ventana del CRUD. Al abrirse llama a Api.Listar y pone el resultado en la tabla; la tabla crea sus columnas sola. Al hacer clic en una fila guardamos su id en idSeleccionado y llenamos las cajas.

> El botón Guardar sirve para dos cosas. Arma un Candidato con lo que hay en las cajas, y la línea clave decide: si idSeleccionado es cero, es nuevo y hacemos POST con Crear; si no, estamos editando y hacemos PUT con Actualizar.

> Después, Mostrar revisa el código de respuesta: éxito, 403 u otro error, y muestra el mensaje que corresponde. Luego recarga la tabla. Eliminar funciona igual, pero con DELETE. Ahora lo vemos funcionando.

Diapositiva 13 · Demostración en vivo

> (Cambiar a la aplicación.) Primero voy a probar el login con una contraseña incorrecta... y la ventana dice Email o contraseña incorrectos: la API respondió 400.

> Ahora con la contraseña correcta... y se abre la ventana de candidatos con la tabla ya llena: eso fue el GET.

> Voy a crear un candidato nuevo: escribo nombres, apellidos, cargo Rector y ámbito Universidad, y presiono Guardar. Dice Candidato guardado, y aparece al final de la tabla: eso fue un POST que devolvió 201.

> Ahora lo selecciono, le cambio el cargo a Decano y el ámbito a Facultad, y Guardar otra vez: eso fue un PUT. Con la fila seleccionada presiono Eliminar, y desaparece: un DELETE con 204.

> Por último entro con el usuario normal, test2. Puedo ver la lista, pero si intento guardar, dice Solo un administrador puede hacer esto: la API respondió 403.

> Como ven, la ventana solo muestra datos; las reglas están en la API y nosotros la consumimos con HTTP y JSON.

Diapositiva 14 · Cuestionario

> Con esto terminamos. Gracias por su atención. Para cerrar, les pedimos que escaneen este código QR y respondan las cinco preguntas del cuestionario sobre lo que acabamos de mostrar. ¿Alguna pregunta?

5. Explicación del código

Dónde está: todo es nuevo, en la carpeta `elecicion-api\cliente-csharp-grupo3\ClienteEleccion`. En la API **no se cambió código**; solo se configura el `.env` (sección 6).

Archivo	Estado	Qué hace
`ClienteEleccion.csproj`	Creado	Define el proyecto: Windows Forms con .NET 10.
`Program.cs`	Creado	Punto de entrada: abre la ventana de login.
`Candidato.cs`	Creado	Clase con Id, Nombres, Apellidos, Cargo y Ambito: el molde del JSON.
`Api.cs`	Creado	Todas las llamadas HTTP a la API.
`FormLogin.cs`	Creado	Ventana de email y contraseña.
`FormCandidatos.cs`	Creado	Ventana con tabla, cuatro cajas de texto y cuatro botones.

Api.cs: las líneas importantes

```
static readonly HttpClient http = new() { BaseAddress = new Uri("http://
127.0.0.1:8000/api/") };
```

Un solo cliente HTTP para toda la aplicación, apuntando a la API. Por eso las demás llamadas solo dicen `"login"` o `"candidatos"`.

```
var resp = await http.PostAsJsonAsync("login", new { email, password });
if (!resp.IsSuccessStatusCode) return false;
```

Hace el POST del login mandando email y contraseña como JSON. Si la API responde con error (400), devuelve `false`.

```
var datos = await resp.Content.ReadFromJsonAsync<RespuestaLogin>();
http.DefaultRequestHeaders.Authorization = new
AuthenticationHeaderValue("Bearer", datos!.Token);
```

Lee el token de la respuesta y lo deja fijo en el encabezado `Authorization: Bearer ...`. Desde aquí todas las peticiones van autorizadas.

```
public static async Task<List<Candidato>> Listar() =>
    await http.GetFromJsonAsync<List<Candidato>>("candidatos") ?? new();
```

GET a `/api/candidatos`; convierte el JSON directo en una lista de objetos `Candidato`.

```
Crear      → http.PostAsJsonAsync("candidatos", c)
Actualizar → http.PutAsJsonAsync($"candidatos/{c.Id}", c)
Eliminar   → http.DeleteAsync($"candidatos/{id}")
```

Los otros tres verbos. Devuelven la respuesta completa para que la ventana revise el código (201, 200, 204 o 403).

FormLogin.cs: las líneas importantes

```
if (await Api.Login(txtEmail.Text, txtPassword.Text))
{
    Hide();
    new FormCandidatos().ShowDialog();
    Close();
}
```

Si el login funciona, esconde el login, abre la ventana del CRUD y, cuando esta se cierra, termina la aplicación. El `try/catch` que lo rodea avisa si la API no está corriendo.

FormCandidatos.cs: las líneas importantes

```
Load += async (s, e) => await Cargar();
async Task Cargar() { tabla.DataSource = await Api.Listar(); }
```

Al abrir la ventana se piden los candidatos y se ponen en la tabla. `DataSource` crea las columnas solo, a partir de las propiedades de `Candidato`.

```
var c = (Candidato)tabla.Rows[e.RowIndex].DataBoundItem;
idSeleccionado = c.Id;
```

Al hacer clic en una fila se recupera el candidato de esa fila, se guarda su id y se llenan las cajas.

```
var resp = idSeleccionado == 0 ? await Api.Crear(c) : await Api.Actualizar(c);
```

La línea clave de Guardar: sin fila elegida hace POST (crear); con fila elegida hace PUT (editar).

```
else if ((int)resp.StatusCode == 403) MessageBox.Show("Solo un administrador
puede hacer esto");
```

En `Mostrar`: traduce el código HTTP a un mensaje para el usuario y luego recarga la tabla.

6. Cómo dejarlo corriendo

API, en la carpeta `elecicion-api`:

```
composer install
copy .env.example .env
php artisan key:generate
```

El `.env.example` de esta carpeta ya trae estas dos líneas. **Ojo:** con `JWT_SECRET=secret` el login falla con "Provided key is too short"; la clave debe tener 32 caracteres o más.

```
JWT_SECRET=clave_secreta_sis258_eleccion_api_2026
JWT_ALGORITHM=HS256
```

Crear la base `eleccion_api` en phpMyAdmin y luego:

```
php artisan migrate:fresh --seed
php artisan serve
```

Cliente, en la carpeta `ClienteEleccion`: `dotnet run`. Usuarios: `test@example.com` (admin) y `test2@example.com` (normal), ambos con contraseña `password`.

7. Preguntas para cuestionarios.sis.com.bo

La respuesta correcta está marcada con (✔).

1. ¿Qué verbo HTTP usa nuestro cliente para crear un candidato?

a. GET
b. POST (✔)
c. PUT
d. DELETE

2. ¿En qué encabezado viaja el token después del login?

a. Content-Type
b. Accept
c. Authorization: Bearer <token> (✔)
d. Cookie

3. ¿Qué responde la API si un usuario que no es administrador intenta eliminar?

a. 200 OK
b. 401 No autorizado
c. 403 Acceso denegado (✔)
d. 404 No encontrado

4. ¿Qué clase de .NET usamos para hacer las peticiones a la API?

a. HttpClient (✔)
b. SqlConnection
c. WebBrowser
d. Socket

5. ¿En qué formato intercambian datos el cliente C# y la API?

a. XML
b. CSV
c. JSON (✔)
d. Texto plano

8. Si el inge pregunta

Pregunta	Respuesta corta
¿Por qué un solo HttpClient?	Para guardar el token una sola vez y reutilizar la conexión; crear uno por petición desperdicia recursos.
¿Qué pasa si se apaga la API?	El `try/catch` del login muestra "No se pudo conectar con la API".
¿Cómo sabe la tabla qué columnas mostrar?	`DataSource` lee las propiedades públicas de `Candidato` y crea una columna por cada una.
¿Diferencia entre 401 y 403?	401: no hay token o es inválido. 403: el token es válido, pero el usuario no tiene permiso.
¿Por qué async/await?	Para que la ventana no se congele mientras espera la respuesta de la red.